Lecture 03 - Part II

Resource Virtualization

From CPU to I/O, storage, and network - the same ideas, different axes

High Performance and Cloud Computing 2025/2026
Applied Data Science & Artificial Intelligence - HPC Track
University of Trieste

Learning Objectives - Part II

By the end of Part II, the student will be able to…

1. Explain the four approaches to I/O virtualization (emulation, virtio, passthrough, SR-IOV) and map each to the Part I CPU techniques.
2. Describe the main virtual-disk formats and the trade-off between thin and thick provisioning.
3. Describe the layered network virtualization stack, and understand why VXLAN is the foundation of cloud VPCs.
4. Reason about virtualization overhead in HPC contexts and identify the tuning levers (CPU pinning, huge pages, SR-IOV, GPU passthrough).
5. Name the modern cloud hypervisor implementations (AWS Nitro, Firecracker) and anticipate their role in Lecture 07.

Outline - Part II

Applying the Popek–Goldberg framework beyond the CPU

9	**I/O virtualization**	*~15 min*
10	**Storage virtualization**	*~8 min*
11	**Network virtualization**	*~10 min*
12	**Performance considerations for HPC**	*~5–7 min*
13	**Closing: AWS Nitro & Firecracker preview**	*~1–2 min*

…some definitions

Hypervisor Architectures

Type 1 vs Type 2

What is a Hypervisor?

A hypervisor (VMM) is the software layer that creates and manages VMs

Core responsibilities:

- CPU scheduling: Allocate physical CPU time to virtual CPUs
- Memory management: Map guest physical memory to host memory
- I/O mediation: Handle virtual device access to physical hardware
- Isolation: Ensure VMs cannot interfere with each other

Key Terminology

Host: The physical machine running the hypervisor

Guest: A virtual machine running on the host

VMM: Virtual Machine Monitor (synonym for hypervisor)

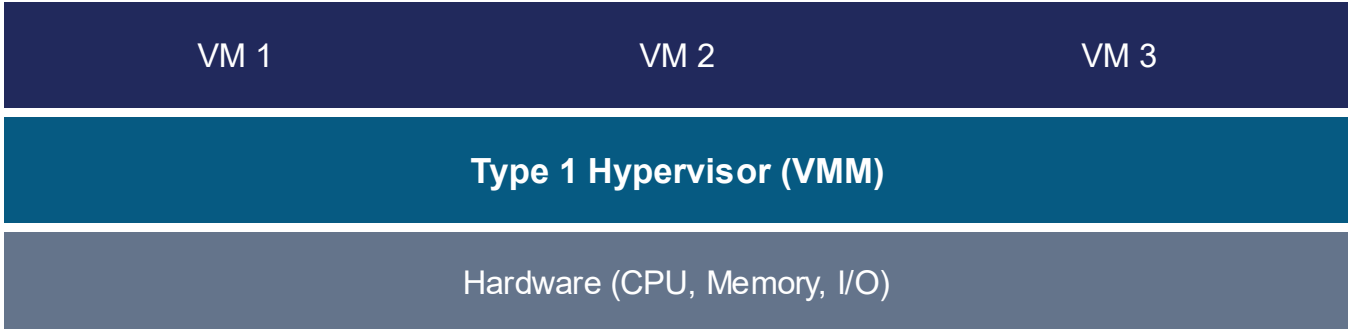
vCPU: Virtual CPU - a CPU core as seen by the guest

Dom0/DomU: Xen terminology: privileged/unprivileged domain

QEMU: Quick Emulator - provides device emulation for KVM

Type 1: Bare-Metal Hypervisor

Runs directly on hardware - no host operating system



Characteristics:

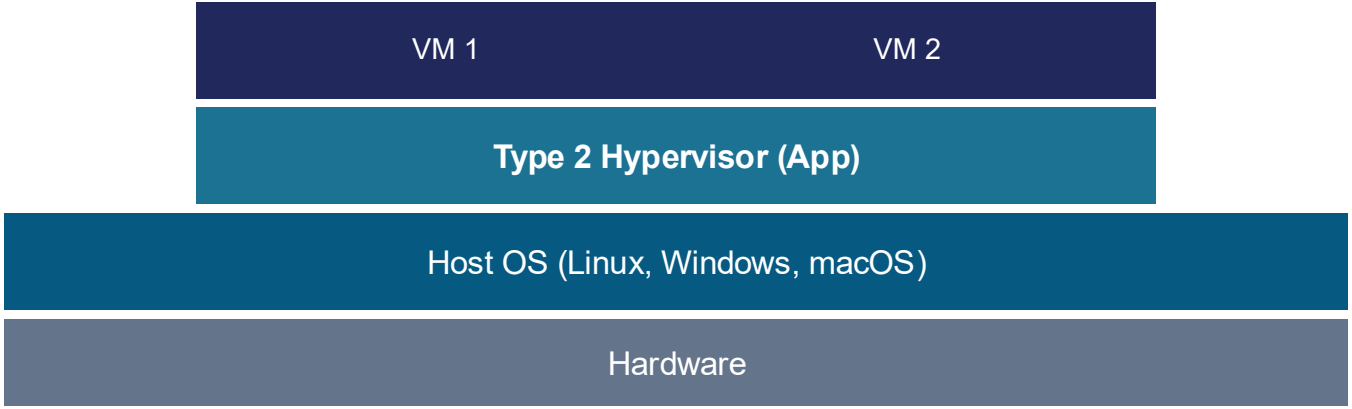
Direct HW access, better performance, smaller attack surface

Examples:

VMware ESXi, Microsoft Hyper-V, Xen, KVM

Type 2: Hosted Hypervisor

Runs as an application on a host operating system



Characteristics:

Easy to install, host OS overhead, good for dev/test

Examples:

VMware Workstation, VirtualBox, Parallels

Type 1 vs Type 2: Comparison

Aspect	Type 1 (Bare-Metal)	Type 2 (Hosted)
Performance	Better (direct HW)	Host OS overhead
Security	Smaller attack surface	Host OS vulnerabilities
Use case	Data centers, cloud	Desktop, development
Setup	Dedicated machine	Install like any app
HW support	Limited (own drivers)	Broad (host drivers)

Cloud providers use Type 1 hypervisors for production

KVM: A Hybrid Approach

KVM turns Linux into a Type 1 hypervisor

How KVM works:

- KVM is a Linux kernel module (merged in 2007)
- Leverages hardware virtualization (VT-x/AMD-V)
- QEMU provides device emulation and VM management
- Linux provides scheduling, memory management, drivers

KVM/QEMU is the foundation of OpenStack, GCP, and many clouds

9

I/O virtualization

The same three answers applied to devices.

From CPU to I/O

The same three answers reappear - on a different axis

Recap from Part I:

- We saw three strategies to virtualize the CPU:
  - binary translation (VMM rewrites code at runtime),
  - paravirtualization (guest is modified to cooperate),
  - hardware-assisted (CPU is extended with VT-x).

The same framework applies to I/O devices.

Devices (NICs, disks, GPUs) were also designed for exclusive access by a single OS. Sharing them among multiple guests raises the same three-way question: software workaround, guest cooperation, or new hardware?

Four approaches to I/O virtualization

Emulation, virtio, passthrough, SR-IOV - mapped to Part I

I/O virtualization — the same three answers, applied to devices

Each I/O technique mirrors one of the CPU virtualization strategies from Part I

slowest → fastest / near-native

increasing performance, decreasing guest flexibility →

Emulation

QEMU: software model of a real device (e.g. e1000 NIC)

Maps to Part I: **Binary translation**

Mechanism:
VMM intercepts every MMIO access, every port-I/O, every DMA descriptor; simulates the device in software.

Pros:
Unmodified guest drivers.
Maximum compatibility (Windows, legacy OSes, any NIC/disk).

Cons:
Very many VMEXITs per I/O.
10–100× slower than native.

HPC verdict:
Unusable for high-performance networking or storage.

virtio

guest & host cooperate via shared-memory ring buffers

Maps to Part I: **Paravirtualization**

Mechanism:
Guest uses virtio drivers that post requests to a shared ring; VMM processes in batch.

Pros:
~10× less VMEXITs than emulation.
Industry standard on KVM.
Used across net, block, SCSI, GPU.

Cons:
Requires virtio-aware guest drivers (available on Linux, Windows via add-ons).

HPC verdict:
Good for general workloads; insufficient for tight MPI.

Passthrough

one physical device assigned exclusively to one VM (via IOMMU)

Maps to Part I: **Hardware-assisted**

Mechanism:
Intel VT-d / AMD-Vi remap DMA and interrupts so a single VM can safely own a real device.

Pros:
Near-native I/O performance.
No VMM involvement on fast path.
Guest uses real device drivers.

Cons:
Device locked to one VM.
Breaks live migration.
Poor density (1 device / tenant).

HPC verdict:
Standard for GPU compute; used for InfiniBand, NVMe.

SR-IOV

one physical function (PF) + many virtual functions (VFs)

Maps to Part I: **HW-assisted + multiplexing**

Mechanism:
The device itself exposes N independent "virtual" instances; each VF is passed through to one VM.

Pros:
Near-native per-VM performance.
Multi-tenant: one NIC → many VMs.
E.g. Intel X710: up to 128 VFs.

Cons:
Requires SR-IOV-capable hardware.
Complex NIC/switch configuration.

HPC verdict:
The cloud-HPC networking solution.
AWS ENA, EFA both use SR-IOV.

The pattern:
The same three strategies we saw for the CPU — software workaround (BT), guest-aware cooperation (paravirt), and hardware support (VT-x) — reappear for I/O as emulation, virtio, and passthrough/SR-IOV. Different axis, same idea.

Emulation

The slowest but most compatible approach

How it works

- VMM presents a software model of a real physical device (e.g. Intel e1000 NIC).
- Every MMIO access, port I/O, DMA descriptor ring, interrupt is simulated in software.
- Guest uses its normal device driver - thinks it's talking to real hardware.
- **Canonical implementation: QEMU.**

Trade-offs

- Guest completely unmodified; maximum compatibility.
- Every I/O requires many VMEXITs - catastrophic for throughput.
- 10–100× slower than native I/O.
- **Unsuitable for production HPC or high-throughput workloads.**

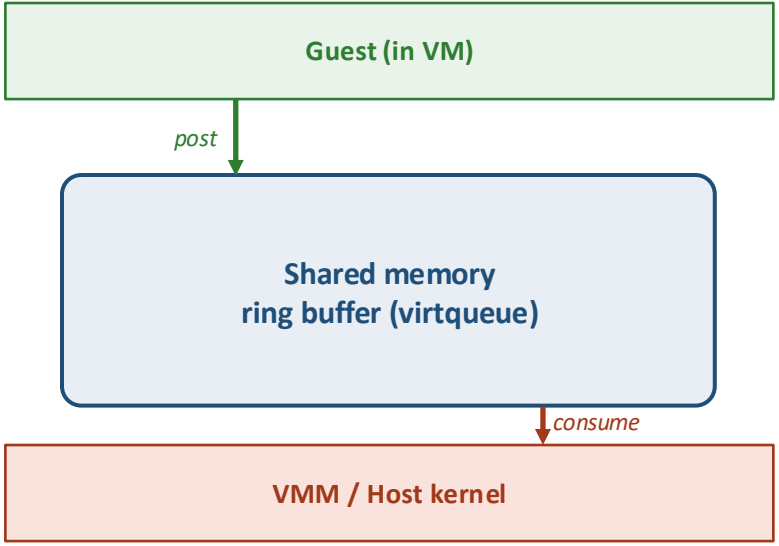
Part I analogue: *Binary translation: the VMM does all the work; guest runs unchanged, but at high cost.*

virtio - paravirtualized I/O

The industry standard for KVM-based virtualization

The ring buffer idea

- Guest and VMM share a memory region - the virtqueue.
- Guest posts I/O requests to the queue and continues.
- VMM processes requests in batch - fewer VMEXITs.
- Completions are signalled via interrupt or polling.
- *Guest drivers are virtio-aware (available on Linux, Windows with add-ons).*



Part I analogue: *Paravirtualization the guest is modified to cooperate; near-native performance at the cost of driver specialization.*

Passthrough and SR-IOV

Hardware-assisted I/O: near-native for one or many guests

Passthrough

How it works

One physical device is dedicated to one VM. Intel VT-d / AMD-Vi (IOMMU) remaps DMA and interrupts so the guest can safely own real hardware.

Pros

Near-native performance. No VMM involvement on fast path.

Cons

Device locked to one VM. Breaks live migration. Poor density.

HPC context

Standard for GPU compute, NVMe, InfiniBand.

SR-IOV

How it works

The device exposes itself as 1 Physical Function + many Virtual Functions. Each VF is passed through to a different VM. Multiplexing is done in hardware, inside the device.

Pros

Near-native per-VM performance + multi-tenancy.

Cons

Requires SR-IOV-capable hardware. Complex config.

HPC context

The cloud-HPC networking solution. AWS ENA & EFA use SR-IOV.

Why I/O matters for HPC

Latency, bandwidth, and the MPI problem

- **HPC workloads are I/O-intensive in very specific ways:**
  - MPI jobs require microsecond-level inter-node latency.
  - GPU workloads move tens of GB/s across PCIe.
  - Parallel file systems (Lustre, BeeGFS) saturate multi-GB/s per node.
- **Emulation and virtio cannot deliver this.**
- Only passthrough and SR-IOV are viable - and they shape the cloud offering:
  - AWS EFA (Elastic Fabric Adapter): SR-IOV NIC bypassing the kernel for MPI, ~15 µs latency.
  - AWS P4/P5 instances: GPU passthrough for training and HPC.
  - Azure HBv3 / HC: InfiniBand passthrough + SR-IOV for tight MPI.

Cloud HPC lives or dies by the quality of I/O virtualization.

I/O virtualization - recap

The same three answers, different devices

	Emulation	virtio	Passthrough	SR-IOV
Part I analogue	*Binary translation*	*Paravirtualization*	*HW-assisted*	*HW-assisted + HW multiplex*
Guest modified?	**No**	**Yes (virtio drivers)**	**No**	**No**
Performance	10–100× slower	~10× better than emulation	Near-native	Near-native
Multi-tenancy?	Yes	Yes	No (1 device : 1 VM)	**Yes (1 device : N VMs)**
HPC verdict	Unsuitable	Good for general use	**GPU, NVMe**	**Networking (EFA)**

10 Storage virtualization

Indirection at the block level.

Storage virtualization

Virtual disks - indirection at the block level

- **A virtual disk is a file (or logical volume) on the host, seen as a block device by the guest.**
- The VMM translates block-level operations:
  - ✓ guest issues sector-N read → VMM maps it to an offset in the host file → host storage responds.
- Same principle as memory virtualization: add a level of indirection.
- Key properties to understand:
  - ✓ disk format - how the virtual disk is represented on the host,
  - ✓ provisioning - how space is allocated (thin vs thick),
  - ✓ backend - where the actual bytes live.

Virtual disk formats

Not all formats are equivalent

Format	Origin	Key features	When to use
QCOW2	KVM / QEMU	Thin provisioning, snapshots, compression, encryption, backing files	Default choice on KVM; supports all advanced features
VMDK	VMware	Snapshots, multiple flavours (sparse, flat, streamOptimized)	VMware ESXi environments; OVF/OVA distribution
VHD / VHDX	Microsoft	VHDX: up to 64 TB, resilient to power failure, logging	Hyper-V, Azure VM disks
RAW	universal	No metadata, 1:1 with the block device. No snapshots, no thin provisioning.	When maximum performance matters and features are not needed

Trade-off: rich formats (QCOW2, VHDX) vs raw performance. Raw wins on throughput; QCOW2 wins on features.

Thin vs thick provisioning

Allocate on demand, or upfront

Thick provisioning

Reserve the full capacity on the host at VM creation time.

Pros

- Predictable performance - no allocation on the fast path.
- No risk of host running out of space unexpectedly.
- Simpler operational model.

Cons

- Wastes host storage when VMs are over-provisioned.
- Slow VM creation for large disks.

Thin provisioning

Allocate host storage only when the guest actually writes.

Pros

- Enables storage overcommitment across VMs.
- Fast VM creation (no upfront allocation).
- Better utilization in typical workloads.

Cons

- Host can run out of storage without warning.
- Write amplification - first-write penalty on each block.
- Requires monitoring + quotas to be safe.

§10.3 · Storage backends

Where the actual bytes live - and why it matters for HPC

Backend	Access pattern	Strengths	HPC relevance
Local disk	Direct-attached to host	Lowest latency, highest IOPS	Scratch, checkpoints
NFS / NAS	File-level over network	Shared, simple	Shared home directories
iSCSI / FC SAN	Block-level over network	Shared, fast, reliable	Enterprise HPC storage
Ceph / GlusterFS	Distributed object / block	Scalable, fault-tolerant	Scale-out cloud storage
Parallel FS (Lustre)	Distributed, parallel striping	GB/s per client, POSIX	**The HPC default**

For HPC workloads with tight I/O bounds, the virtualization layer should stay out of the way: direct passthrough to Lustre/BeeGFS or fast local NVMe.

11

Network virtualization

From vNICs to cloud VPCs.

Network virtualization

A layered stack - and the foundation of cloud VPCs

- Virtualization inside a single host gives us isolated VMs, but:
  - ✓ VMs need to communicate - with each other, with the host, with the external world.
  - ✓ Cloud deployments need network isolation between tenants and between workloads.
- **The solution is a stack of network virtualization primitives, each adding capability:**
  - ✓ vNIC - give each VM a (virtual) network card.
  - ✓ vSwitch - software switch connecting vNICs.
  - ✓ VLAN - segment traffic at layer 2.
  - ✓ Overlay networks (VXLAN) - scale beyond the limits of VLANs, cross-subnet.
  - ✓ SDN - programmatically orchestrate all of the above.

The network virtualization stack

Five layers, each building on the previous

5. SDN - software-defined networking

Centrally program all layers via APIs. The control plane of modern clouds.

4. Overlay networks (VXLAN)

L2 frames encapsulated in UDP. 24-bit VNI → 16M tenants. Cross-L3. Foundation of cloud VPCs.

3. VLANs (802.1Q)

L2 segmentation inside a switch. 12-bit VLAN ID → max 4094 networks per physical L2.

2. Virtual switches (Linux Bridge, Open vSwitch)

Software switch inside the host connecting vNICs and uplinks.

1. Virtual NICs (vNICs)

Each VM gets a virtual network interface (often virtio-net or SR-IOV VF).

VXLAN: overlay networks in one slide

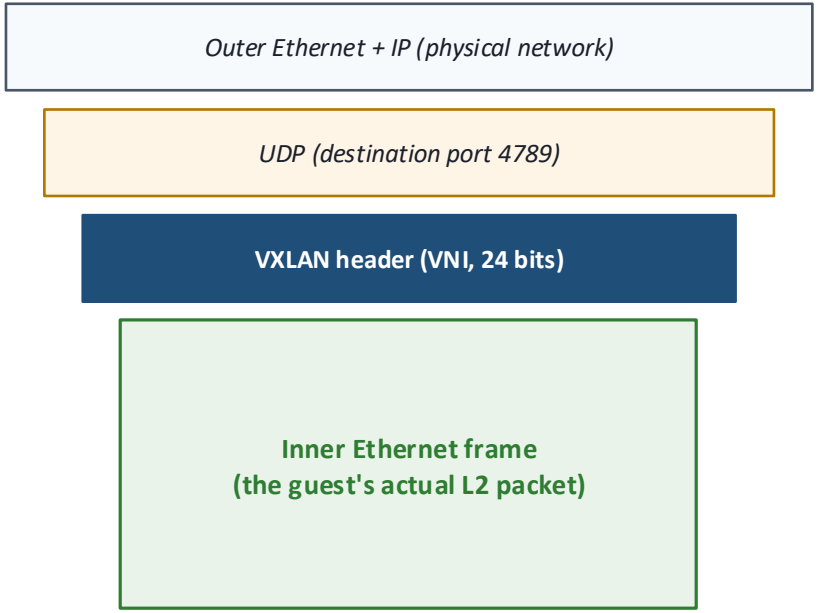
Why cloud VPCs are built on top of this

The problem with VLANs

- Only 4094 VLAN IDs (12 bits).
- Bounded to a single L2 domain - cannot span across subnets.
- Insufficient for clouds with thousands of tenants.

The VXLAN idea

- Encapsulate L2 frames inside UDP packets (VXLAN header + UDP + IP).
- 24-bit VNI → 16M virtual networks.
- L2 over L3 - works across any IP routing.
- **Tenant isolation is by VNI, not by physical topology.**



Encapsulation

From VXLAN to cloud VPCs

How AWS VPC, Azure VNet, GCP VPC actually work

- **Cloud Virtual Private Clouds (VPCs) = VXLAN-based overlays + SDN control plane.**
- Each VPC = a VXLAN VNI (or equivalent tenant identifier).
- Multiple VPCs can coexist on the same physical network - complete L2/L3 isolation.
- Cross-region / cross-AZ connectivity uses the overlay to route over any underlay.
- Subnets, routing tables, security groups - all programmed via SDN APIs on top of the VXLAN fabric.
- **Example: AWS VPC**
  - custom encapsulation similar to VXLAN (not exactly VXLAN on the wire, but same idea);
  - hardware-offloaded on AWS Nitro cards - zero CPU cost on the host;
  - SDN control plane via AWS APIs: create subnet, modify route, attach internet gateway.

Understanding VXLAN means understanding why VPCs work the way they do.

12

Performance considerations for HPC

Where the overhead hides, and how to tame it.

§12 · Where the overhead comes from

Five sources - all addressable with tuning

1. **VMEXITs**

   Each transition non-root → root costs hundreds of cycles. Minimize by choosing HW-assisted + paravirt I/O.

2. **Memory 2D walks**

   EPT/NPT double the worst-case TLB miss cost (up to 4×4 accesses on x86-64). Mitigate with huge pages.

3. **I/O emulation**

   Pure emulation is 10–100× slower than native. Use virtio, passthrough, or SR-IOV.

4. **Scheduling effects**

   vCPU preemption by host scheduler breaks MPI synchrony. Use CPU pinning and dedicated hosts.

5. **Interrupt delivery**

   Virtual interrupts add latency. Posted interrupts (VT-x) and direct-assigned IRQs help.

Well-tuned: 2–5% overhead. Poorly tuned: 20%+. *The difference is configuration, not virtualization.*

HPC-specific tuning

The levers that matter for scientific workloads

CPU

CPU pinning + NUMA awareness

Bind vCPUs to physical cores on the same NUMA node as their memory. Prevents cross-socket traffic.

Critical for MPI, OpenMP. Can be 2–5× performance difference.

MEM

Huge pages (2 MB, 1 GB)

Reduce TLB pressure on 2D page walks. Especially important for large-memory scientific codes.

5–15% improvement on memory-bound HPC workloads.

NET

SR-IOV for networking

Essential for InfiniBand, RoCE, AWS EFA. Bypasses the VMM on the fast path.

Reduces MPI latency from ~100µs (virtio) to ~5–15µs (SR-IOV).

GPU

GPU passthrough

Assign a full GPU to a guest via VT-d/IOMMU. No multi-tenancy, near-native GPU performance.

The default for ML training and scientific GPU compute.

Virtualization is not free - but it is good enough

HPC in the cloud: 95–98% of bare-metal performance, achievable

With the right tuning:

95–98%

of bare-metal performance for typical HPC workloads

- The answer is not "avoid virtualization" - it's "configure it correctly".
- Modern cloud HPC offerings (AWS HPC, Azure HBv3, Google C2) ship with the right defaults out of the box.
- **Understanding the overhead sources is what separates a working HPC cloud deployment from a struggling one.**

A 60-second preview - AWS Nitro and Firecracker

Modern hypervisors rethought. Full treatment in Lecture 07.

AWS Nitro (2017)	Firecracker (2018)
Offloads virtualization (network, storage, management) to custom silicon cards. The host CPU is dedicated almost entirely to the guest.	A microVM - minimal KVM-based hypervisor. Boots in <125ms, memory footprint <5MB. Written in Rust for security.
• Near-zero virtualization overhead • Powers all modern EC2 instances • Enables bare-metal EC2 instances	• Powers AWS Lambda and Fargate • Container-like density, VM-level isolation • Purpose-built for serverless

Lecture 07 will cover both in depth alongside AWS EC2 instance types.

Some Definitions

To know for the final exam ☺

Virtualization

*It is the ability to **simulate a hardware platform**, such as a server, storage device or network resource, in software. All of the functionality is separated (abstracted) from the hardware and simulated as a "**virtual instance**" with the ability to operate just like the hardware solution. A single hardware platform can be used to support multiple virtual devices or machines, which are easy to spin up or down as needed.*

Virtual Machine

It is the software simulation of a computer. It is able to run an Operating Systems and applications interacting with the virtualized abstracted resources, not with the physical resources, of the actual host computer.

Hypervisor (or Virtual Machine Monitor)

It is a software tool installed on the physical host system to provide the thin software layer of abstraction that decouples the OS from the physical bare-metal. It allows to split a computer in different separate environment, the Virtual Machines, distributing them the computer resources

Virtualization Features

Five layers, each building on the previous

Consolidation

Run different OSes on the same physical host Partition physical resources between VMs

Isolation

Fault and security isolation at the hardware level Preserve performance with advanced resource control

Encapsulation

Save the VM state to files VMs can be moved and copied moving and copying files (migration)

Hardware Independence

VMs can be copied, moved or migrated to different physical servers

Other

Debugging, profiling, reproducibility, software preservation

Recap - Part II

The whole lecture in six points

1. **Virtualization extends beyond the CPU.**
   Memory, I/O, storage, network all need virtualization - each raises the same three-way question.
2. **I/O: emulation, virtio, passthrough, SR-IOV.**
   The four approaches mirror Part I's BT / paravirt / HW-assisted - applied to devices.
3. **Storage = indirection at block level.**
   Formats (QCOW2, RAW) and provisioning (thin vs thick) shape the trade-off. Backend matters for HPC.
4. **Network is a layered stack.**
   vNIC → vSwitch → VLAN → VXLAN → SDN. Cloud VPCs are VXLAN overlays with SDN control planes.
5. **Virtualization is not free, but overhead is tameable.**
   With CPU pinning, huge pages, SR-IOV, GPU passthrough: 95–98% of bare metal for HPC.
6. **Modern clouds rethink the hypervisor (Nitro, Firecracker).**
   Hardware offload and purpose-built microVMs are the next evolution. More in Lecture 07.

End of Lecture 03

Questions?

→ **Next: Appendix - Memory virtualization (warm-up to Lecture 04)**

Shadow page tables, EPT/NPT, memory overcommitment (~30 min)